

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №2

по дисциплине

«Технологии компонентного программирования»

«СОМ-сервер: калькулятор и его клиенты»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Герман Олег Витольдович
кандидат технических наук
доцент кафедры ИТАС

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	2
2	Теоретические сведения.....	3
2.1	Компонентный подход	3
2.2	Модель COM	3
2.3	Реестр и активация	4
2.4	Позднее связывание и коды ошибок.....	5
2.5	COM-сервер на Python	5
3	Компонентное программирование в Linux и macOS в сравнении с Windows	6
3.1	Windows	6
3.2	Linux	6
3.3	macOS	7
3.4	Реестр и его аналоги	7
3.5	Сводное сравнение	8
3.6	Следствие для данной работы.....	8
4	Реализация	10
4.1	Структура программы.....	10
4.2	COM-класс	10
4.3	Регистрация	10
4.4	Сервер.....	11
4.5	Клиент	11
4.6	Графическое приложение.....	12
4.7	Клиент в MS Excel.....	12
4.8	Интеграция в HTML-страницу.....	13
4.9	Вариант для Windows	13
5	Результаты работы программы	14
6	Ответы на контрольные вопросы	19
	Заключение	20
	Список использованных источников.....	21

1 ЦЕЛЬ РАБОТЫ

Изучить компонентную объектную модель COM: понятия COM-объекта, интерфейса, идентификаторов CLSID и ProgID, регистрацию и активацию сервера. Создать на языке Python COM-сервер, выполняющий базовые алгебраические операции (сложение, вычитание, умножение, деление и возведение в степень), зарегистрировать его и продемонстрировать использование из нескольких клиентов: графического калькулятора, табличного процессора MS Excel и HTML-страницы. Дополнительно сравнить средства компонентного программирования в Windows, Linux и macOS.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Компонентный подход

Компонент — это самостоятельный блок программы, который обладает тремя свойствами:

- интерфейс отделён от реализации: клиенту виден только набор методов;
- компонент поставляется в готовом (собранном) виде, а не исходным кодом;
- компонент находится и подключается во время выполнения по имени, а не на этапе сборки клиента.

От обычного класса или библиотеки компонент отличается именно последнее свойство: клиент и компонент могут быть написаны на разных языках, разными людьми и в разное время, а связывает их только описание интерфейса. Это даёт повторное использование, замену реализации без пересборки клиентов, изоляцию сбоев и расширяемость за счёт подключаемых модулей. Цена подхода — накладные расходы на вызов через границу процесса, сложность развёртывания (компонент нужно установить и зарегистрировать) и жёсткие требования к неизменности интерфейсов.

2.2 Модель COM

COM (Component Object Model) — технология Microsoft, появившаяся в 1993 году как основа OLE. Это двоичный стандарт: он описывает не язык программирования, а то, как объект выглядит в памяти и как к нему обращаться. Основные понятия приведены в таблице 2.1.

Таблица 2.1 – Основные понятия COM

Понятие	Смысл
COM-объект	экземпляр COM-класса, реализующий один или несколько интерфейсов
Интерфейс	набор методов, через которые клиент работает с объектом; сам объект клиенту недоступен
IUnknown	базовый интерфейс любого объекта: запрос других интерфейсов (QueryInterface) и подсчёт ссылок (AddRef, Release)
IDispatch	универсальный интерфейс для скриптовых клиентов: вызов метода по его имени
CLSID	128-битный идентификатор класса (GUID), уникальный в мире
ProgID	читаемое имя класса, например Lab2.Calculator
HRESULT	32-битный код результата вызова; ошибки возвращаются кодом, а не исключением языка

2.3 Реестр и активация

Связь «имя класса → код, который его реализует» хранит не клиент и не сервер, а операционная система — в разделе реестра HKEY_CLASSES_ROOT. При регистрации сервера создаются ключи, показанные в листинге 2.1.

Листинг 2.1 – Ключи реестра COM-сервера

```
HKEY_CLASSES_ROOT\Lab2.Calculator\CLSID      = {7C1B9A52-...}
HKEY_CLASSES_ROOT\CLSID\{7C1B9A52-...}\ProgID  = Lab2.Calculator
HKEY_CLASSES_ROOT\CLSID\{7C1B9A52-...}\LocalServer32 = pythonw.exe -m win32com.server.localserver {7C1B9A52-...}
```

Обращение клиента к компоненту проходит четыре шага:

1. **регистрация** — выполняется один раз, как правило с правами администратора;
2. **активация** — клиент вызывает CreateObject или Dispatch с именем класса; система по ProgID находит CLSID, по CLSID — команду запуска и запускает сервер;
3. **вызов** — клиент обращается к методам через интерфейс;
4. **освобождение** — когда счётчик ссылок обнуляется, объект уничтожается, а сервер завершается.

По расположению различают три вида серверов: внутрипроцессный (DLL загружается в адресное пространство клиента, ключ InprocServer32), локальный (отдельный процесс на той же машине, ключ LocalServer32) и удалённый (другая машина, технология DCOM). В работе используется локальный сервер, как и в методических материалах (CLSCTX_LOCAL_SERVER).

2.4 Позднее связывание и коды ошибок

Скриптовые клиенты (VBA, Python, JScript) не знают устройства интерфейса на этапе компиляции, поэтому работают через IDispatch. Вызов метода разбивается на два обращения: GetIDsOfNames переводит имя метода в числовой идентификатор DISPID, а Invoke выполняет метод с этим идентификатором и набором аргументов. Такой способ называется поздним связыванием: имя метода является строкой и проверяется во время выполнения.

Коды ошибок, которые встречаются в работе, приведены в таблице 2.2.

Таблица 2.2 – Коды HRESULT, используемые в работе

Имя	Значение	Когда возникает
CO_E_CLASSSTRING	0x800401F3	ProgID не зарегистрирован
DISP_E_UNKNOWNNAME	0x80020006	у объекта нет метода с таким именем
DISP_E_EXCEPTION	0x80020009	метод завершился исключением (деление на ноль)

2.5 COM-сервер на Python

Библиотека `pywin32` позволяет объявить COM-класс обычным классом Python с набором служебных атрибутов: `_reg_progid_` и `_reg_clsid_` задают имя и идентификатор, `_reg_clsctx_` — вид сервера, а `_public_methods_` перечисляет методы, доступные клиентам. Регистрация выполняется запуском скрипта с ключом `--register`, отмена — с ключом `--unregister`. Идентификатор CLSID следует сгенерировать один раз и записать в класс: если получать его вызовом `uuid.uuid4()` при каждом запуске, в реестре накапливаются записи, которые уже ни на что не указывают.

3 КОМПОНЕНТНОЕ ПРОГРАММИРОВАНИЕ В LINUX И MACOS В СРАВНЕНИИ С WINDOWS

3.1 Windows

В Windows компонентная модель является частью операционной системы. COM служит основой для целого семейства технологий: OLE и ActiveX (внедрение объектов и элементы управления), OLE Automation (управление приложениями Office из скриптов), DCOM и COM+ (распределённые объекты и транзакции), оболочка Windows и DirectX. Более поздние платформы эту модель не заменили, а надстроили: сборки .NET умеют выступать COM-объектами и вызывать их, а среда Windows Runtime построена на COM-интерфейсах.

Характерные черты: единый двоичный стандарт для всех языков, единый системный реестр, системная служба активации, которая сама запускает сервер по обращению клиента.

3.2 Linux

В Linux единой компонентной модели, встроенной в систему, нет. Те же задачи решаются несколькими независимыми механизмами.

D-Bus — шина сообщений для вызовов между процессами и ближайший аналог COM по назначению. Сервис занимает на шине имя вида `org.freedesktop.Notifications`, публикует объекты с путями, интерфейсами и методами. Клиент на любом языке вызывает метод по имени сервиса, пути объекта и имени метода. Если сервис не запущен, шина запускает его сама по файлу описания в каталоге `/usr/share/dbus-1/services` — это то же, что ключ `LocalServer32`. Стандартный интерфейс `org.freedesktop.DBus.Introspectable` возвращает описание методов объекта, что соответствует позднему связыванию через `IDispatch`.

Разделяемые библиотеки и подключаемые модули. Роль внутрипроцессного сервера играют файлы `.so`, загружаемые функцией `dlopen`. Объектную модель поверх них дают библиотеки рабочих столов: `GObject` с механизмом `GObject Introspection` в GNOME, подключаемые модули Qt и KParts в KDE.

Модели отдельных приложений. LibreOffice содержит собственную компонентную модель UNO, Mozilla — XPCOM, устроенную по образцу COM. Общим стандартом в 1990-е годы была CORBA (на ней строилась модель Bonobo в GNOME), но сегодня она вытеснена D-Bus для локальных вызовов и протоколами REST и gRPC для сетевых.

3.3 macOS

В macOS компонентные механизмы тоже распределены по нескольким технологиям.

Apple Events и AppleScript — аналог OLE Automation. Приложение публикует словарь своих объектов и команд, а скрипт управляет им: `tell application "Microsoft Excel"`. Именно так на macOS автоматизируются приложения Office, Finder и Safari.

ХРС — современный механизм вызовов между процессами. Сервисы описываются файлами настроек и запускаются системной службой `launchd` по первому обращению, то есть активация по требованию присутствует, как и в COM.

Пакеты и подключаемые модули. Внутрипроцессные компоненты оформляются как библиотеки `.dylib`, фреймворки и пакеты (`bundle`). Механизм `CFPlugIn` из Core Foundation построен по образцу COM: в нём используются интерфейс `IUnknown`, идентификаторы `UUID` и функция-фабрика.

3.4 Реестр и его аналоги

Единого реестра ни в Linux, ни в macOS нет. Его функции выполняют каталоги с файлами описаний (таблица 3.1). Отличие принципиальное: в Windows компонент должен записать себя в центральную базу, а в Linux и macOS достаточно положить файл описания в известный каталог, и система находит компонент сама.

Таблица 3.1 – Где хранится связь «имя → программа»

Задача	Linux	macOS
Имя сервиса → команда запуска	файлы <code>.service</code> в <code>/usr/share/dbus-1/services</code> , модули <code>systemd</code>	файлы <code>plist</code> службы <code>launchd</code>
Тип файла → приложение	файлы <code>.desktop</code> в <code>/usr/share/applications</code>	база <code>Launch Services</code> , <code>Info.plist</code> приложения
Настройки программ	текстовые файлы в <code>/etc</code> и <code>~/.config</code> , база <code>dconf</code>	файлы <code>plist</code> в <code>~/Library/Preferences</code>

3.5 Сводное сравнение

Таблица 3.2 – Средства компонентного программирования в трёх системах

Задача	Windows	Linux	macOS
Вызов объекта в другом процессе	COM, локальный сервер	D-Bus	XPC, Apple Events
Управление приложением из скрипта	OLE Automation, VBA	интерфейсы D-Bus, UNO	AppleScript, JXA
Компонент внутри процесса	DLL, InprocServer32	.so, GObject, модули Qt	.dylib, пакеты, CFPlugIn
Запуск сервера по обращению	служба активации COM	активация D-Bus, systemd	launchd
Поиск компонента по имени	реестр	каталоги файлов описаний	файлы plist, Launch Services
Вызов по имени метода	IDispatch	интроспекция D-Bus	словари сценариев
Удалённый вызов	DCOM	gRPC, REST	gRPC, REST
Единый стандарт для системы	да	нет	нет

Из сравнения следуют три вывода. Во-первых, компонентное программирование не привязано к Windows: во всех трёх системах есть вызов объекта в другом процессе по имени, активация по требованию и подключаемые модули. Во-вторых, Windows отличается не набором возможностей, а тем, что они собраны в одну модель с одним реестром; в Linux и macOS для каждой задачи свой механизм. В-третьих, сами COM-объекты между системами не переносятся: Excel для macOS не может создать объект по ProgID, а библиотека `ruwin32` не устанавливается вне Windows.

3.6 Следствие для данной работы

Работа выполнялась на компьютере с macOS, где COM отсутствует. Поэтому реализация разделена на две части. Класс калькулятора написан один раз и оформлен по правилам `ruwin32`. Для Windows к нему прилагается скрипт настоящей регистрации. Для macOS и Linux написана переносимая модель COM, которая воспроизводит механизм: реестр с теми же ключами, запуск сервера отдельным процессом, вызов через `GetIDsOfNames` и `Invoke`, коды `HRESULT`. Соответствие частей показано в таблице 3.3.

Таблица 3.3 – Настоящий COM и переносимая модель

Часть	Windows	Модель в работе
Реестр	HKEY_CLASSES_ROOT	файл artifacts/registry.json с теми же ключами
Регистрация	pywin32, ключ --register	registry.register
Процесс сервера	win32com.server.localserver	python -m lab2.localserver
Клиент	win32com.client.Dispatch, CreateObject	lab2.client.Dispatch
Транспорт	системный механизм COM	строки JSON через локальный сокет

Модель не является COM: в ней нет интерфейса IUnknown и подсчёта ссылок, передачи указателей на интерфейсы между процессами и библиотек типов, а посторонние программы (Excel, Word) объект модели не видят.

4 РЕАЛИЗАЦИЯ

4.1 Структура программы

Листинг 4.1 – Структура лабораторной работы

lab2/calculator.py	COM-класс Calculator: пять операций
lab2/registry.py	реестр: регистрация и отмена регистрации
lab2/localserver.py	процесс сервера: GetIDsOfNames, Invoke, Release
lab2/client.py	Dispatch и прокси-объект, ошибки ComError
lab2/web.py	HTTP-сервер и HTML-страница
lab2/cases.py	проверяемые сценарии
lab2/gui.py	окно: калькулятор, COM-объект, HTML-клиент
lab2/runner.py	запуск сценариев из командной строки
lab2/windows/calc_com_server.py	регистрация через pywin32 (Windows)
lab2/windows/calc_com_client.py	клиент на pywin32 (Windows)
lab2/windows/CalculatorClient.bas	макрос VBA для Excel и Word (Windows)

4.2 COM-класс

Класс калькулятора (листинг 4.2) не зависит от операционной системы. Список `_public_methods_` определяет интерфейс компонента. Метод `secret` существует в Python, но в список не входит, поэтому клиенту недоступен — так проверяется, что наружу видно только объявленное.

Листинг 4.2 – COM-класс калькулятора

```
class Calculator:
    _reg_progid_ = "Lab2.Calculator"
    _reg_clsids_ = "{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}"
    _reg_desc_ = "Lab 2 calculator COM server"
    _reg_clsctx_ = CLSCTX_LOCAL_SERVER
    _public_methods_ = ["Add", "Sub", "Mul", "Div", "Pow"]

    def Add(self, a: float, b: float) -> float | int:
        return _tidy(float(a) + float(b))

    def Div(self, a: float, b: float) -> float | int:
        if float(b) == 0:
            raise ZeroDivisionError("division by zero")
        return _tidy(float(a) / float(b))

    def Pow(self, a: float, b: float) -> float | int:
        return _tidy(math.pow(float(a), float(b)))
```

4.3 Регистрация

Функция `class_keys` (листинг 4.3) формирует те же ключи, которые `pywin32` записывает в реестр Windows: имя класса указывает на идентификатор, идентификатор — на команду запуска сервера и на класс Python. Регистрация добавляет эти ключи в файл, отмена регистрации удаляет их.

Листинг 4.3 – Ключи, создаваемые при регистрации

```
def class_keys(cls: type) -> dict[str, str]:
    progid, clsid = cls._reg_progid_, cls._reg_clsids_
```

```

command = shlex.join([sys.executable, "-m", "lab2.localserver", clsid])
return {
    rf"HKCR\{progid}": cls._reg_desc_,
    rf"HKCR\{progid}\CLSID": clsid,
    rf"HKCR\CLSID\{clsid}": cls._reg_desc_,
    rf"HKCR\CLSID\{clsid}\ProgID": progid,
    rf"HKCR\CLSID\{clsid}\LocalServer32": command,
    rf"HKCR\CLSID\{clsid}\PythonCOM":
        f"{cls.__module__}.{cls.__name__}",
}

```

4.4 Сервер

Сервер запускается отдельным процессом командой из ключа LocalServer32. Он находит класс по CLSID, создаёт объект и обслуживает запросы клиента (листинг 4.4). Идентификатор DISPID — это порядковый номер метода в списке `_public_methods_`. Любое исключение метода превращается в код `DISP_E_EXCEPTION`, обращение к необъявленному имени — в `DISP_E_UNKNOWNNAME`. По запросу Release процесс завершается.

Листинг 4.4 – Обработка запросов в процессе сервера

```

def handle(target: object, request: dict) -> dict:
    public = list(target._public_methods_)
    op = request.get("op")
    if op == "GetIDsOfNames":
        name = request["name"]
        if name not in public:
            return {"hr": DISP_E_UNKNOWNNAME, "error": f"Unknown name: {name}"}
        return {"hr": S_OK, "dispid": public.index(name) + 1}
    if op == "Invoke":
        dispid = request["dispid"]
        if not 1 <= dispid <= len(public):
            return {"hr": DISP_E_MEMBERNOTFOUND, "error": f"Member not found: {dispid}"}
        try:
            result = getattr(target, public[dispid - 1])(*request.get("args", []))
        except Exception as exc:
            return {"hr": DISP_E_EXCEPTION, "error": str(exc)}
        return {"hr": S_OK, "result": result}

```

4.5 Клиент

Функция Dispatch (листинг 4.5) повторяет действия системы при активации: по ProgID находит CLSID, по CLSID — команду запуска. Если имени в реестре нет, возвращается ошибка `CO_E_CLASSSTRING`. Прокси-объект перехватывает обращение к любому атрибуту и превращает вызов `calc.Add(2, 3)` в пару запросов `GetIDsOfNames` и `Invoke`.

Листинг 4.5 – Активация объекта по имени

```

def Dispatch(progid: str, trace: Trace | None = None) -> DispatchProxy:
    clsid = registry.query(rf"HKCR\{progid}\CLSID")
    if clsid is None:
        raise ComError(CO_E_CLASSSTRING,

```

```

        f"Invalid class string: {progid} (ProgID is not registered)")
    command = registry.query(rf"HKCR\CLSID\{clsid}\LocalServer32")
    if command is None:
        raise ComError(CO_E_CLASSSTRING, f"No LocalServer32 for {clsid}")
    return DispatchProxy(progid, clsid, command, trace)

```

4.6 Графическое приложение

Окно написано на библиотеке wxPython и содержит три вкладки, общий журнал вызовов и список проверяемых сценариев.

Вкладка **Calculator** — кнопочный калькулятор. Сам он ничего не вычисляет: каждая операция передаётся COM-объекту (листинг 4.6), а под клавиатурой виден журнал обращений к серверу.

Листинг 4.6 — Калькулятор делегирует вычисление COM-объекту

```

def _object(self) -> DispatchProxy:
    if self.calc is None:
        self.calc = Dispatch(cases.PROGID, self._trace)
        self._refresh()
    return self.calc

def _invoke(self, method: str, a: float | int, b: float | int) -> float | int:
    return getattr(self._object(), method)(a, b)

```

Вкладка **COM object** показывает содержимое реестра и позволяет вручную пройти жизненный цикл объекта кнопками Register, Unregister, Dispatch и Release. Вкладка **HTML client** отображает веб-страницу, работающую с тем же объектом.

4.7 Клиент в MS Excel

Для Windows подготовлен макрос VBA (листинг 4.7), который создаёт объект функцией CreateObject и заполняет лист результатами.

Листинг 4.7 — Макрос VBA для Excel

```

Sub FillSheetFromComServer()
    Dim calc As Object
    On Error Resume Next
    Set calc = CreateObject("Lab2.Calculator")
    On Error GoTo 0
    If calc Is Nothing Then
        MsgBox "Failed to create COM object. Please ensure it is registered."
        Exit Sub
    End If
    Set ws = ThisWorkbook.Sheets(1)
    ws.Range("A2:E2").Value = Array(2, "+", 3, calc.Add(2, 3), "Lab2.Calculator.Add")
    ws.Range("A6:E6").Value = Array(2, "^", 10, calc.Pow(2, 10), "Lab2.Calculator.Pow")
End Sub

```

Excel для macOS создавать объекты по ProgID не умеет, поэтому на macOS роль клиента выполняет сценарий на Python: он получает результаты пяти операций от COM-объекта, записывает их в файл `artifacts/com_calculator.csv` и открывает его в Excel.

4.8 Интеграция в HTML-страницу

Браузер не может создать COM-объект, поэтому между страницей и объектом находится серверное приложение. В методических материалах для этого предлагается Flask либо встроенный HTTP-сервер Python; выбран второй вариант, чтобы не добавлять зависимость. Страница отправляет запрос `/calc?op=Pow&a=2&b=10`, обработчик создаёт COM-объект, вызывает метод и возвращает результат в формате JSON (листинг 4.8).

Листинг 4.8 – Серверная часть HTML-клиента

```
def calculate(op: str, a: float, b: float) -> dict[str, object]:
    reply = {"progid": Calculator._reg_progid_, "clsid": Calculator._reg_clsids_}
    if op not in Calculator._public_methods_:
        return reply | {"error": f"Unknown operation: {op}"}
    try:
        com_object = Dispatch(Calculator._reg_progid_)
    except ComError as exc:
        return reply | {"error": str(exc)}
    try:
        return reply | {"result": getattr(com_object, op)(a, b)}
    except ComError as exc:
        return reply | {"error": str(exc)}
    finally:
        com_object.Release()
```

4.9 Вариант для Windows

Скрипт `lab2/windows/calc_com_server.py` (листинг 4.9) регистрирует тот же класс средствами `pywin32`. Этот вариант на компьютере автора не запускался, так как требует Windows.

Листинг 4.9 – Регистрация настоящего COM-сервера

```
import pythoncom
import win32com.server.register

from lab2.calculator import Calculator as PortableCalculator

class Calculator(PortableCalculator):
    _reg_clsctx_ = pythoncom.CLSCTX_LOCAL_SERVER

if __name__ == "__main__":
    print("Registering COM server with CLSID:", Calculator._reg_clsids_)
    win32com.server.register.UseCommandLine(Calculator)
```

5 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Окно запускается командой `python main.py --lab 2 --gui`. На рисунке 5.1 показан калькулятор после вычисления 12×12 : в журнале под клавиатурой видны обращения к серверу, справа — список сценариев, все девять выполнены успешно.

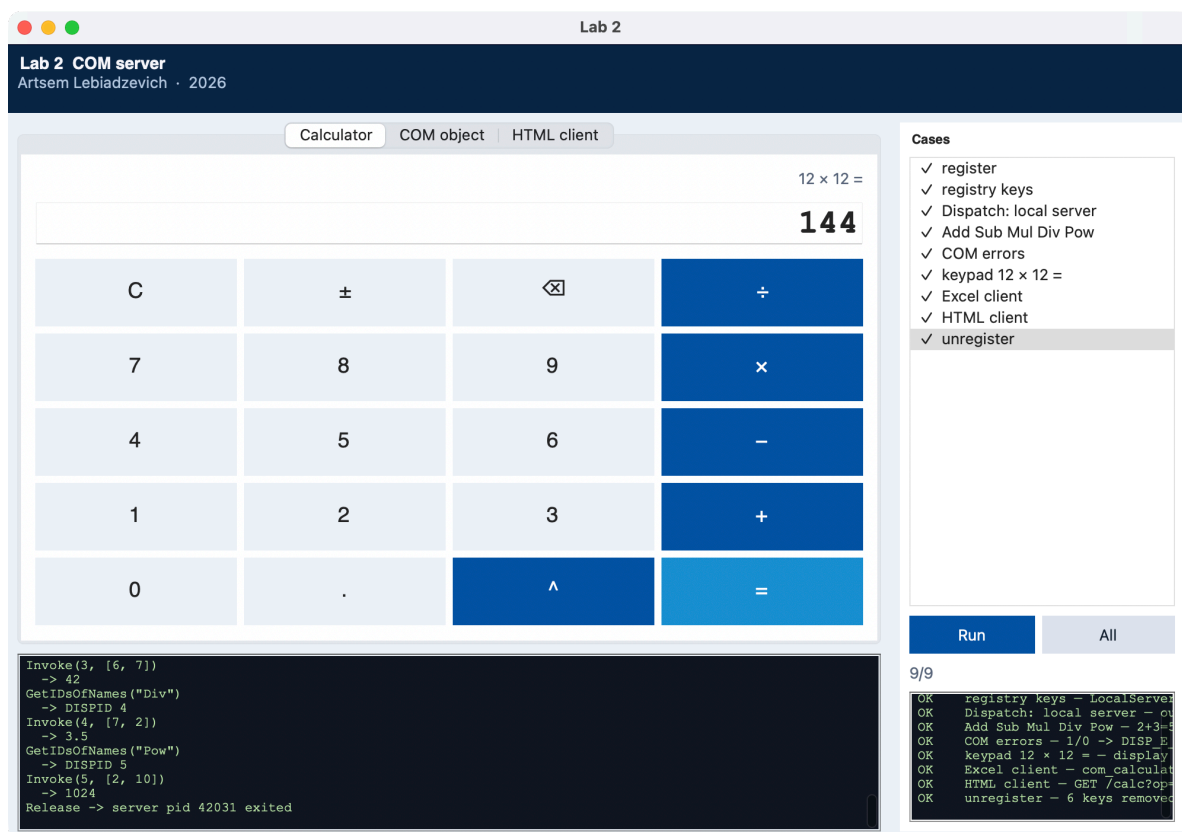


Рисунок 5.1 – Калькулятор, каждая операция которого выполняется COM-объектом

Деление на ноль (рисунок 5.2) приводит к исключению в процессе сервера. Клиент получает его в виде кода `DISP_E_EXCEPTION` с текстовым описанием.

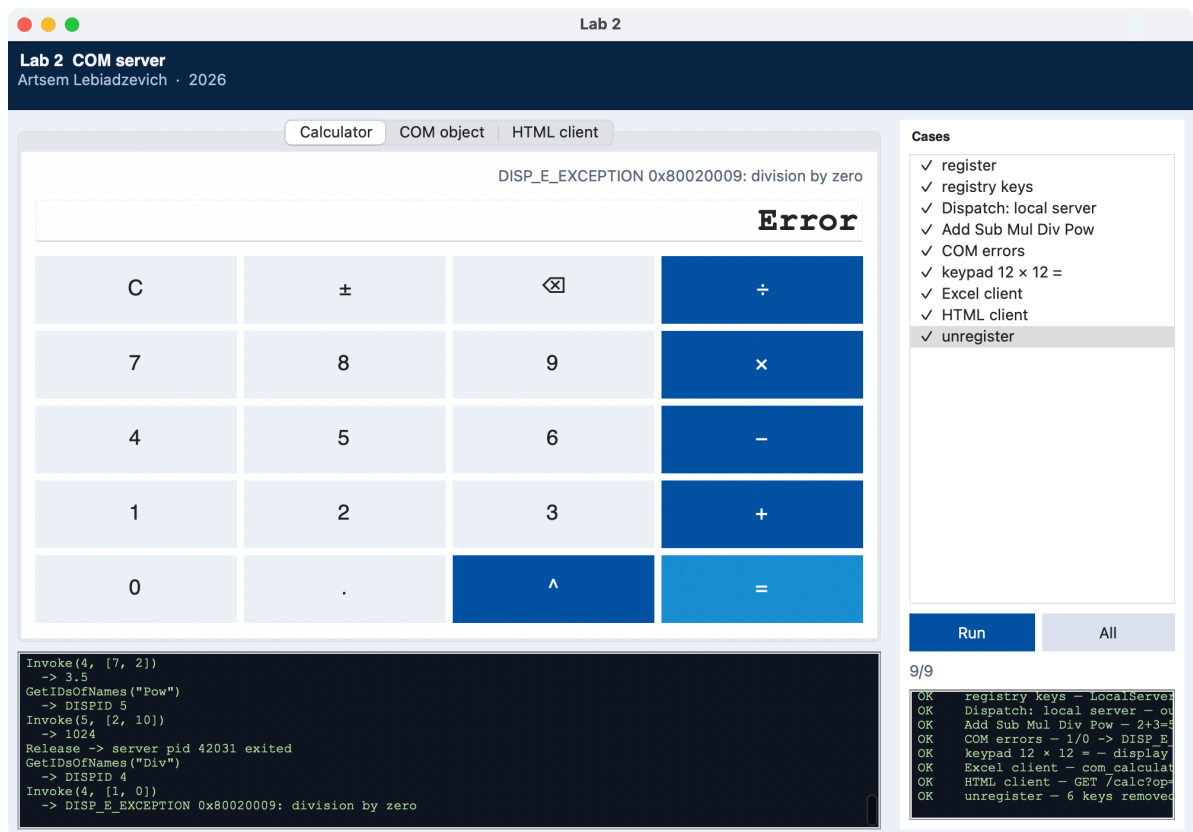


Рисунок 5.2 – Ошибка сервера, переданная клиенту кодом HRESULT

На рисунке 5.3 показаны ключи реестра после регистрации и состояние объекта: класс зарегистрирован, сервер работает в отдельном процессе.

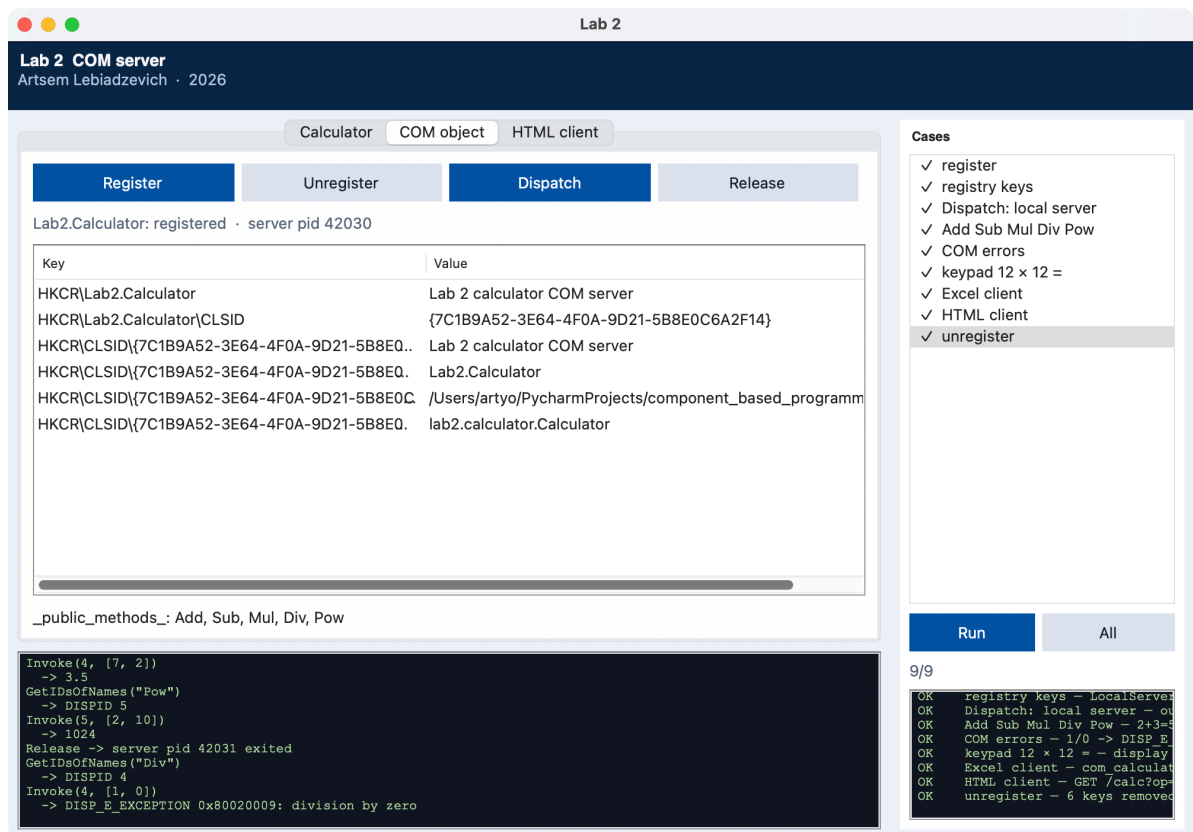


Рисунок 5.3 – Ключи реестра и управление жизненным циклом объекта

HTML-клиент (рисунок 5.4) вычисляет 2^{10} через тот же COM-объект и выводит имя и идентификатор класса.

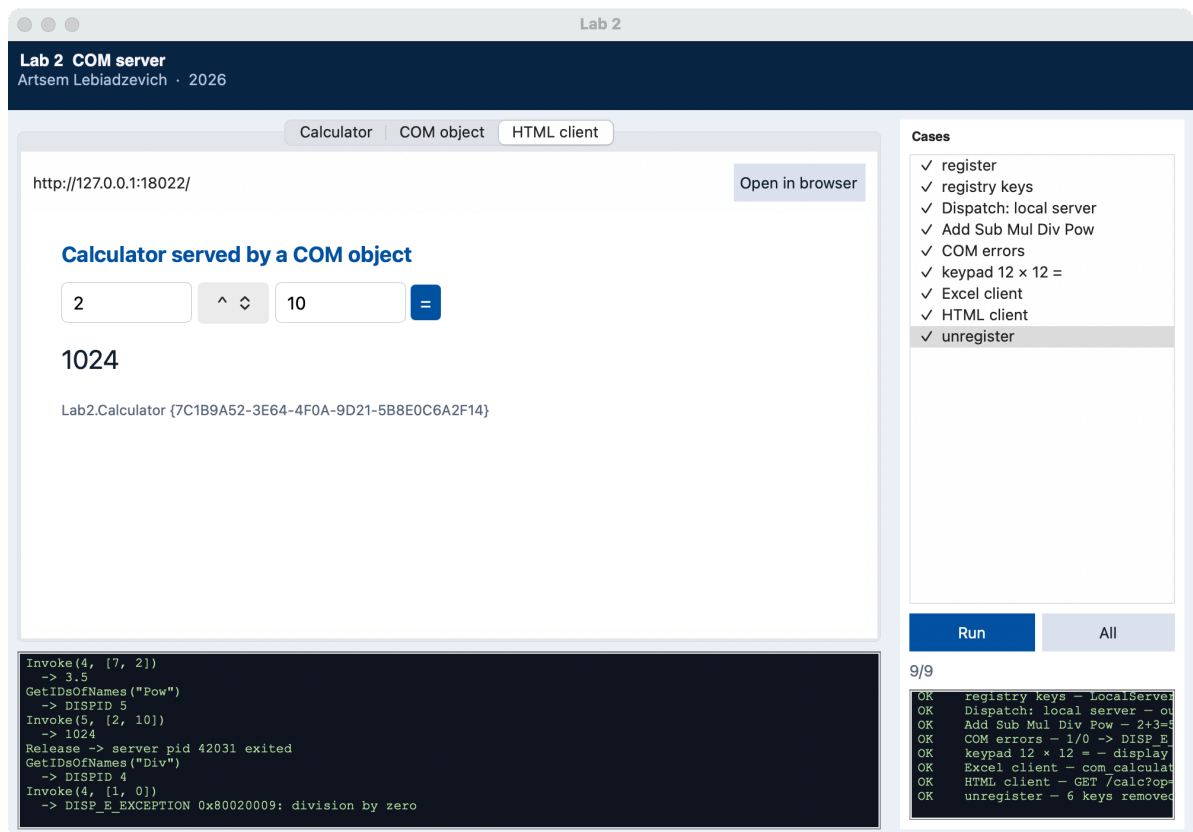


Рисунок 5.4 – HTML-страница, работающая с COM-объектом через серверное приложение

Команда `python main.py --lab 2` выполняет те же сценарии без окна. Её вывод приведён в листинге 5.1: сначала ключи реестра, затем журнал вызовов и итоги сценариев.

Листинг 5.1 – Вывод программы

```
HKCR\Lab2.Calculator = Lab 2 calculator COM server
HKCR\Lab2.Calculator\CLSID = {7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}
HKCR\CLSID\{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14} = Lab 2 calculator COM server
HKCR\CLSID\{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}\ProgID = Lab2.Calculator
HKCR\CLSID\{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}\LocalServer32 = .venv/bin/python -m lab2.localserver '{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}'
HKCR\CLSID\{7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}\PythonCOM = lab2.calculator.Calculator
CoCreateInstance {7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14} -> LocalServer32 pid 42174, port 52441
GetIDsOfNames("Add")
-> DISPID 1
Invoke(1, [2, 3])
-> 5
GetIDsOfNames("Sub")
-> DISPID 2
Invoke(2, [7, 10])
-> -3
GetIDsOfNames("Mul")
-> DISPID 3
Invoke(3, [6, 7])
-> 42
GetIDsOfNames("Div")
-> DISPID 4
Invoke(4, [7, 2])
-> 3.5
GetIDsOfNames("Pow")
-> DISPID 5
Invoke(5, [2, 10])
-> 1024
Release -> server pid 42174 exited
```

```

register -> Lab2.Calculator -> {7C1B9A52-3E64-4F0A-9D21-5B8E0C6A2F14}, 6 keys
dispatch -> out-of-process server pid 42173
arithmetic -> 2+3=5 7-10=-3 6*7=42 7/2=3.5 2^10=1024
com_errors -> 1/0 -> DISP_E_EXCEPTION, secret -> DISP_E_UNKNOWNNAME
excel_sheet -> com_calculator.csv, 5 rows
html_client -> GET /calc?op=Pow&a=2&b=10 -> 1024
unregister -> 6 keys removed, Dispatch -> CO_E_CLASSSTRING; registered again

```

Сводка по сценариям приведена в таблице 5.1.

Таблица 5.1 – Проверяемые сценарии

Сценарий	Что проверяется	Итог
register	ProgID указывает на CLSID	да
registry keys	CLSID указывает на команду запуска и обратно на ProgID	да
Dispatch	сервер работает в другом процессе	да
Add Sub Mul Div Pow	пять операций дают 5, -3, 42, 3,5 и 1024	да
COM errors	деление на ноль и скрытый метод дают коды ошибок	да
keypad	нажатия клавиш $12 \times 12 =$ дают 144	да
Excel client	таблица заполнена результатами объекта	да
HTML client	запрос /calc возвращает 1024	да
unregister	после отмены регистрации Dispatch даёт CO_E_CLASSSTRING	да

Кроме сценариев окна, написаны автоматические тесты: они проверяют, что сервер выполняется в другом процессе, пять операций, три кода ошибок и ответ HTML-клиента. Все тесты проходят.

6 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое COM-сервер и для чего он используется? COM-сервер — программный компонент, который предоставляет объекты через интерфейсы COM, так что к ним могут обращаться программы на других языках и из других процессов. Применяется для автоматизации приложений Office, межпроцессного взаимодействия и повторного использования существующих компонентов.

2. Что такое параметр CLSID и как его создать? CLSID — 128-битный идентификатор COM-класса (GUID). Создаётся один раз, например функцией `uuid.uuid4()`, утилитой `guidgen` или функцией `pythoncom.CreateGuid()`, и после этого не меняется.

3. В каком разделе реестра находится CLSID? В разделе `HKEY_CLASSES_ROOT\CLSID`. Этот раздел является объединённым представлением ветвей `Software\Classes` из `HKEY_LOCAL_MACHINE` и `HKEY_CURRENT_USER`. На идентификатор указывает ключ `HKEY_CLASSES_ROOT\<ProgID>\CLSID`.

4. Как интегрировать COM-сервер в веб-приложение? Браузер создавать COM-объекты не может, поэтому используется серверное приложение (Flask или встроенный HTTP-сервер): оно создаёт объект функцией `Dispatch`, вызывает его метод и возвращает результат странице в виде HTML или JSON.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучена компонентная объектная модель COM и создан COM-сервер на языке Python, выполняющий сложение, вычитание, умножение, деление и возведение в степень. Сервер используется тремя клиентами: графическим калькулятором, таблицей Excel и HTML-страницей, причём ни один из них не импортирует код калькулятора — все обращаются к объекту по имени `Lab2.Calculator`.

Главный практический вывод состоит в том, что клиент компонента знает только имя, а связь имени с кодом хранит система. Поэтому регистрация является обязательным отдельным шагом: при отсутствии записи в реестре исправный и доступный код сервера клиенту недоступен, и активация завершается ошибкой `CO_E_CLASSSTRING`.

Сравнение систем показало, что компонентный подход не ограничен Windows. В Linux вызовы между процессами и активацию по требованию обеспечивает шина D-Bus, в macOS — XPC и Apple Events, а роль реестра выполняют каталоги с файлами описаний. Отличие Windows в том, что эти возможности объединены в одну модель с единым реестром. Сами COM-объекты между системами не переносятся, поэтому на macOS работа выполнена на переносимой модели COM, а вариант с библиотекой `pywin32` и макросом VBA подготовлен для запуска в Windows.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Герман, О. В. СОМ-сервера: лекция и лабораторная работа по дисциплине «Технологии компонентного программирования» / О. В. Герман. — Минск: БГУИР, 2024.
2. Роджерсон, Д. Основы СОМ / Д. Роджерсон. — М.: Русская редакция, 2000. — 400 с.
3. Бокс, Д. Сущность технологии СОМ / Д. Бокс. — СПб.: Питер, 2001. — 400 с.
4. Szyperski, C. Component Software: Beyond Object-Oriented Programming / C. Szyperski. — 2nd ed. — Boston: Addison-Wesley, 2002. — 624 p.
5. Hammond, M. Python Programming on Win32 / M. Hammond, A. Robinson. — Sebastopol: O'Reilly, 2000. — 652 p.
6. D-Bus Specification [Электронный ресурс]. — Режим доступа: <https://dbus.freedesktop.org/doc/dbus-specification.html>.